
Detailed Analysis of the Barcode Reader Docker Files

This document explains the purpose and interaction of the three files: Dockerfile, docker-compose.yml, and barcode_reader.py. The Dockerfile and docker-compose.yml are explained line by line. The Python program is also analysed because it is the application that the container runs.

The listing of the codes can be found in section “**9. Appendices**”.

Edited by: Dr Tan Kwee Teck

Date: 03 Aug 2026

Table of Contents

Table of Contents.....	2
1. Overall Architecture	3
2. Dockerfile — Line-by-Line Explanation.....	4
3. docker-compose.yml — Line-by-Line Explanation	9
4. barcode_reader.py — Functional Analysis.....	12
Imports.....	12
Camera setup	12
Main loop.....	12
Barcode handling.....	12
Duplicate suppression.....	12
GUI code	12
Shutdown.....	12
5. How the Three Files Work Together	13
6. Important Observations and Possible Simplifications	13
7. Commands for building the Docker image, running the container, and stopping/removing the container.....	14
8. Building and Running Without docker-compose.yml	15
8.1 Build the Docker Image.....	15
8.2 Verify the Image.....	15
8.3 Allow Local X11 Access	15
8.4 Run the Container — Multiple-Line Format	15
8.5 Run the Container — Single-Line Format.....	15
8.6 docker run Options and Explanations	16
8.7 Why python3 barcode_reader.py Is Not Added.....	16
8.8 Stop, Restart, and Remove the Container	16
8.9 Typical Manual Workflow.....	17
9. Appendices	18
9.1 Dockerfile	18
9.2 docker-compose.yml.....	20
9.3 barcode_reader.py.....	21

1. Overall Architecture

The Dockerfile builds a 32-bit ARM Debian Bullseye image and installs Python, Raspberry Pi camera software, OpenCV, pyzbar, ZBar, and X11-related packages. The docker-compose.yml builds or selects that image and starts a privileged container with access to Raspberry Pi devices, camera-related system services, and the host X11 display. barcode_reader.py starts the Pi Camera, captures frames, decodes barcodes, and prints a newly detected barcode to the terminal.

2. Dockerfile — Line-by-Line Explanation

Lines 1 to 5: #####
Raspberry Pi 4
Raspberry Pi OS Bullseye (32-bit)
#####

Describe that the intended target is a Raspberry Pi 4, and the host operating system is Raspberry Pi OS Bullseye, 32-bit.

Line 6: FROM arm32v7/debian:bullseye

Selects the base image. arm32v7/debian:bullseye is a 32-bit ARM Debian Bullseye image, suitable for a Raspberry Pi running a 32-bit ARM user space.

Line 7: [blank line]

Line 8: ENV DEBIAN_FRONTEND=noninteractive

Sets an environment variable so Debian package installation does not open interactive configuration prompts. This is useful during unattended Docker builds.

Line 9: [blank line]

Line 10: WORKDIR /app

Sets /app as the working directory. Later COPY and CMD instructions operate relative to this directory unless an absolute path is used.

Line 11: [blank line]

Lines 12 to 14: #####
Install Python

Comment separator.

Line 15: [blank line]

Line 16: RUN apt-get update && \

Starts a shell command during image build. apt-get update downloads the current package index. The backslash continues the RUN command onto the next line for readability.

Line 17: apt-get install -y \

Installs packages without asking for confirmation.

Line 18: python3 \

Installs the Python 3 interpreter.

Line 19: python3-pip \

Installs pip, the Python package installer.

Line 20: python3-dev \

Installs Python development headers and tools needed when compiling some Python extensions.

Line 21: python3-setuptools \

Installs **setuptools**, commonly used by Python package builds.

Line 22: python3-wheel \

Installs wheel support for Python package distribution/builds.

Line 23: && rm -rf /var/lib/apt/lists/*

Continues the same RUN command and deletes downloaded APT package lists to reduce image size.

Line 24: [blank line]

Lines 25 to 27: #####

Add Raspberry Pi Repository

#####

Section label only.

Line 28: [blank line]

Line 29: RUN apt-get update && \

Refreshes package indexes again before installing repository-management tools.

Line 30: apt-get install -y wget gnupg ca-certificates

Installs wget for downloading files, gnupg for key handling, and CA certificates for validating HTTPS certificates.

Line 31: [blank line]

Line 32: RUN wget https://archive.raspberrypi.org/debian/raspberrypi.gpg.key -O /tmp/rpi.gpg && \

Downloads the Raspberry Pi repository signing key to /tmp/rpi.gpg, then continues with the next command.

Line 33: `apt-key add /tmp/rpi.gpg`

Adds the downloaded key to APT's trusted key store. This allows APT to verify packages from the Raspberry Pi repository. Note: apt-key is deprecated in newer Debian releases, although it may still work on Bullseye.

Line 34: [blank line]

Line 35: `RUN echo "deb http://archive.raspberrypi.org/debian bullseye main" > /etc/apt/sources.list.d/raspi.list`

Creates a new APT source-list file containing the Raspberry Pi Bullseye repository. The > operator overwrites or creates the file.

Line 36: [blank line]

Lines 37 to 39: #####
Install Raspberry Pi Camera Packages

Section label only.

Lines 40 to 41: [blank lines]

Line 42: `RUN apt-get update && \`

Refreshes package indexes after adding the Raspberry Pi repository.

Line 43: `apt-get install -y \`

Starts non-interactive installation of the listed packages.

Line 44: `python3-picamera2 \`

Installs Picamera2, the Python interface used by barcode_reader.py.

Line 45: `python3-libcamera \`

Installs Python bindings for libcamera.

Line 46: `python3-opencv \`

Installs OpenCV Python bindings used for colour conversion and optional display functions.

Line 47: python3-pyzbar \

Installs the pyzbar Python package from Debian. The program imports decode from this package.

Line 48: libcamera0 \

Installs the libcamera shared library.

Line 49: libcamera-apps \

Installs libcamera command-line applications.

Line 50: libcamera-tools \

Installs libcamera utility tools.

Line 51: libzbar0 \

Installs the ZBar shared library used underneath pyzbar to decode barcodes and QR codes.

Line 52: libgtk-3-0 \

Installs GTK 3 runtime libraries, which may be required by GUI-related applications.

Line 53: libcanberra-gtk3-module \

Installs a GTK 3 sound/event module; it may be pulled in for GUI desktop compatibility.

Line 54: x11-apps \

Installs small X11 test applications. These are useful for checking whether the container can display windows on the host X server.

Line 55: && rm -rf /var/lib/apt/lists/*

Deletes APT package lists to reduce the final image size.

Lines 56 to 58: [blank lines]

Lines 59 to 61: #####
Copy source

Section label only.

Line 62: [blank line]

Line 63: COPY . .

Copies the entire Docker build context into the current working directory, /app. Because the build context is ., this normally copies Dockerfile, docker-compose.yml, barcode_reader.py, and any other non-excluded files. A .dockerignore file can prevent unwanted files from being copied.

Line 64: [blank line]

Line 65: CMD ["python3","barcode_reader.py"]

Instruct the container to run the "barcode_reader.py" program.

3. docker-compose.yml — Line-by-Line Explanation

Line 1: version: "3.9"

Declares the Compose file format version as 3.9. Modern Docker Compose implementations may treat this field as obsolete and infer the Compose specification automatically, but it is still commonly seen.

Line 2: [blank line]

Line 3: services:

Begins the services section. Each named service below describes one container configuration.

Line 4: [blank line]

Line 5: barcode_nowindow:

Defines a service named barcode_nowindow. This is the Compose service name and is used in commands such as docker compose up barcode_nowindow.

Line 6: image: barcode_nowindow:v1

Specifies the image name and tag to use: barcode_nowindow:v1. After a build, Compose tags the resulting image with this name.

Line 7: build: .

Tells Compose to build the image using the current directory (.) as the build context. Compose looks for a file named Dockerfile in that directory unless told otherwise.

Line 8: [blank line]

Blank line used to separate sections and improve readability.

Line 9: container_name: barcode_nowindow

Sets the explicit runtime container name to barcode_nowindow. Without this, Compose normally generates a project-based name.

Line 10: [blank line]

Blank line used to separate sections and improve readability.

Line 11: privileged: true

Runs the container in privileged mode. This grants very broad access to host devices and kernel capabilities. It is useful for hardware experiments but is much less isolated than a normal container.

Line 12: [blank line]

Blank line used to separate sections and improve readability.

Line 13: network_mode: host

Uses the host network namespace. The container shares the Raspberry Pi host's network interfaces and ports. Port mappings are therefore unnecessary and should not be used with this mode.

Line 14: [blank line]

Line 15: `stdin_open: true`

Keeps standard input open even when the container is not attached. This is equivalent to `docker run -i` and can help with interactive use.

Line 16: [blank line]

Line 17: `tty: true`

Allocates a pseudo-terminal. This is equivalent to `docker run -t` and makes terminal output/interaction behave more like a normal shell.

Line 18: [blank line]

Line 19: `restart: "no"`

Disables automatic restart. If the program exits or the container stops, Docker will not restart it automatically.

Line 20: [blank line]

Line 21: `environment:`

Begins the environment-variable section.

Line 22: `- DISPLAY=${DISPLAY}`

Passes the host `DISPLAY` environment variable into the container. `DISPLAY` tells X11 GUI programs which display server to use. The `${DISPLAY}` value is expanded by Compose from the host environment.

Line 23: [blank line]

Line 24: `volumes:`

Begins the bind-mount section.

Line 25: `- /tmp/.X11-unix:/tmp/.X11-unix`

Mounts the host X11 Unix-domain socket directory into the container at the same path. Together with `DISPLAY` and suitable X-server permissions, this allows GUI applications in the container to open windows on the host desktop.

Line 26: `- /dev:/dev`

Mounts the entire host `/dev` directory into the container. This exposes hardware device nodes, including camera-related nodes. Because privileged mode is already enabled, this is broad access and may be more than the application needs.

Line 27: `- /run/udev:/run/udev:ro`

Mounts the host udev runtime directory read-only. This can allow software in the container to inspect device information while preventing writes to that host directory.

Line 28: - /run/dbus:/run/dbus

Mounts the host D-Bus runtime directory. This may be needed by some system or camera-related components that communicate through D-Bus. It is read-write here because :ro is not specified.

Line 29: [blank line]

Line 30: devices:

Begins the devices section.

Line 31: - /dev/video0

Explicitly passes /dev/video0 into the container. This is a narrower device mapping, but it is partly redundant when the whole /dev directory is already mounted and privileged mode is enabled.

Line 32: [blank line]

4. barcode_reader.py — Functional Analysis

This Python program is the application launched by the Dockerfile. It is supposed to be a GUI version, but to minimize potential problem some lines of code that involve the GUI have been commented. The `cv2.imshow()`, `cv2.waitKey()`, barcode rectangle, and barcode text code are commented out. Therefore, the active program prints detected barcode information to the terminal.

If you want to try with GUI version, just remove all the `'''` in the Python file (lines 41, 49, 55, 67, 83 and 88).

Imports

Picamera2 provides Raspberry Pi camera access. `cv2` is OpenCV and is used here for RGB-to-BGR conversion. `pyzbar.decode` sends an image to the ZBar decoder.

Camera setup

`Picamera2()` creates the camera object. `create_preview_configuration()` requests a 640×480 main stream. `configure()` applies the configuration, and `start()` begins camera operation.

Main loop

`while True` continuously captures frames. `capture_array()` returns an image array. The code converts RGB to BGR because OpenCV conventionally uses BGR ordering. `decode(frame)` returns all barcodes found in the frame.

Barcode handling

For each detected barcode, `barcode.rect` provides x, y, width, and height. The rectangle-drawing code is disabled. `barcode.data` is decoded from bytes to UTF-8 text, and `barcode.type` identifies the barcode format.

Duplicate suppression

`last_barcode` stores the most recently printed barcode. The program prints only when the current barcode differs from `last_barcode`, preventing repeated terminal output while the same barcode remains in view. A consequence is that scanning barcode A, then B, then A will print A again, but removing A and presenting A again without scanning another barcode will not print it.

GUI code

The display window, key handling, and q-to-quit logic are commented out. Thus “Press q to quit.” is currently misleading: q is not being read. The active program runs indefinitely unless the process/container is stopped externally. If you delete all the `'''` in the Python file (lines 41, 49, 55, 67, 83 and 88), then the GUI will be included, and you can press ‘q’ (when the video window is active/selected) to quit the program.

Shutdown

`picam2.stop()` is placed after the infinite “while” loop, but the active loop has no break statement (because of lines 83 and 88 commenting out the ‘q’ key-press function to break the “while” loop). Therefore, this line is normally unreachable unless the GUI codes are uncommented. It would run only if the loop were changed to terminate normally.

5. How the Three Files Work Together

1. **docker compose build** reads docker-compose.yml. The build: . setting sends the current directory as the build context and uses the Dockerfile.
2. The Dockerfile creates the ARM32 Debian image, installs the required software, copies the application files into /app, and sets the default command.
3. **docker compose up** creates the barcode_nowindow container using the image barcode_nowindow:v1.
4. Compose grants broad hardware access and mounts the X11, device, udev, and D-Bus paths.
5. The container runs python3 barcode_reader.py.
6. The Python program accesses the camera, captures 640×480 frames, decodes barcodes, and prints new barcode values.

6. Important Observations and Possible Simplifications

- a) The service is called barcode_nowindow, but DISPLAY and the X11 socket are still configured. This is harmless if retained for testing, but they are not needed by the current Python code because its GUI calls are commented out. If you decide to include the GUI, then DISPLAY and the X11 socket are needed.
- b) The message “Press q to quit.” does not match the active code. The program will loop forever. You may either remove the message, or uncomment the waitKey()/q logic.
- c) privileged: true, /dev:/dev, and devices: /dev/video0 provide overlapping hardware access. The configuration works as a broad-access setup, but it is more permissive than necessary. A future hardening exercise could identify the minimum required devices and mounts.
- d) network_mode: host is not required by the current barcode program unless another part of the project needs host networking.
- e) COPY . . may copy unnecessary files into the image. The .dockerignore file (also provided in the repo) reduces image size and avoid copying temporary files.
- f) The Dockerfile runs apt-get update several times. This is understandable because a repository is added between package-install stages, but some layers could be consolidated to reduce image layers.
- g) The Raspberry Pi repository key is installed with apt-key, which is deprecated. A keyring-based approach is preferable for newer Debian practices.
- h) Because the program has no active exit path, stopping the container is the normal way to end it. A try/finally block could ensure picam2.stop() runs during controlled shutdown.

7. Commands for building the Docker image, running the container, and stopping/removing the container

Build:

```
docker compose build
```

Start in the foreground:

```
docker compose up
```

Start in the background:

```
docker compose up -d
```

View logs:

```
docker compose logs -f
```

Stop and remove the container:

```
docker compose down
```

Rebuild after changing the Dockerfile or source:

```
docker compose up --build
```

8. Building and Running Without docker-compose.yml

The Docker image and container can be built and run directly with Docker commands. The docker-compose.yml file is not required when the equivalent build and runtime settings are supplied explicitly on the command line.

8.1 Build the Docker Image

Open a terminal in the directory containing the Dockerfile and run:

```
docker build -t barcode_nowindow:v1 .
```

8.2 Verify the Image

Run the following command to verify that the image was created:

```
docker image ls
```

8.3 Allow Local X11 Access

If the container will run an X11 GUI application, execute this on the Raspberry Pi desktop before starting the container:

```
xhost +local:
```

For the build command, -t assigns the image name barcode_nowindow and tag v1, while the final dot (.) specifies the current directory as the build context. The current barcode_reader.py has its OpenCV GUI code commented out, so the X11 settings are not required for terminal-only barcode output.

8.4 Run the Container — Multiple-Line Format

The backslash (\) at the end of each line tells the shell that the command continues on the next line.

```
docker run \  
    --name barcode_nowindow \  
    --privileged \  
    --network host \  
    -it \  
    -e DISPLAY=$DISPLAY \  
    -v /tmp/.X11-unix:/tmp/.X11-unix \  
    -v /dev:/dev \  
    -v /run/udev:/run/udev:ro \  
    -v /run/dbus:/run/dbus \  
    --device /dev/video0 \  
    barcode_nowindow:v1
```

8.5 Run the Container — Single-Line Format

You can also run the container using single-line format command, without using the backslash (\).

```
docker run --name barcode_nowindow --privileged --network host -it -e DISPLAY=$DISPLAY  
-v /tmp/.X11-unix:/tmp/.X11-unix -v /dev:/dev -v /run/udev:/run/udev:ro -v  
/run/dbus:/run/dbus --device /dev/video0 barcode_nowindow:v1
```

8.6 docker run Options and Explanations

The table below explains the purpose of the options used in the docker command to run the container.

Option / Argument	Compose Equivalent	Explanation
--name barcode_nowindow	container_name	Sets the explicit container name.
--privileged	privileged: true	Gives broad access to host devices and kernel capabilities; useful for hardware access but less isolated.
--network host	network_mode: host	Shares the Raspberry Pi host network namespace.
-i	stdin_open: true	Keeps standard input open.
-t	tty: true	Allocates a pseudo-terminal.
-it	stdin_open + tty	Combines interactive input and a terminal.
-e DISPLAY=\$DISPLAY	environment	Passes the host DISPLAY variable into the container for X11 GUI applications.
-v /tmp/.X11-unix:/tmp/.X11-unix	volumes	Shares the host X11 socket so container GUI windows can appear on the host desktop.
-v /dev:/dev	volumes	Exposes the complete host /dev directory; this overlaps with --privileged and --device.
-v /run/udev:/run/udev:ro	volumes	Provides read-only access to host udev runtime information.
-v /run/dbus:/run/dbus	volumes	Shares the host D-Bus runtime directory.
--device /dev/video0	devices	Explicitly passes the video device into the container; partly redundant with /dev:/dev and privileged mode.
barcode_nowindow:v1	image	Specifies the Docker image used to create the container.

8.7 Why python3 barcode_reader.py Is Not Added

The Dockerfile already defines CMD ["python3", "barcode_reader.py"]. Docker therefore runs the Python program automatically when the container starts, unless another command is supplied.

8.8 Stop, Restart, and Remove the Container

The table below lists some commonly used Docker commands to stop or delete Docker containers.

Docker Command	What the command does
docker stop barcode_nowindow	Stop the running container.
docker start -ai barcode_nowindow	Restart the stopped container. Note that a container is not deleted when it is stopped. You can check this with command <code>docker ps -a</code>
docker rm barcode_nowindow	Delete the stopped container. If the container has not been stopped yet before you issue this command, you may see an error.
docker rm -f barcode_nowindow	Delete the container even if it has not been stopped. This command will stop the container, and then delete it.
docker rmi barcode_nowindow:v1	Delete the Docker image.

8.9 Typical Manual Workflow

```
cd ~/your-project-directory

docker build -t barcode_nowindow:v1 .

xhost +local:

docker run \
    --name barcode_nowindow \
    --privileged \
    --network host \
    -it \
    -e DISPLAY=$DISPLAY \
    -v /tmp/.X11-unix:/tmp/.X11-unix \
    -v /dev:/dev \
    -v /run/udev:/run/udev:ro \
    -v /run/dbus:/run/dbus \
    --device /dev/video0 \
    barcode_nowindow:v1
```

9. Appendices

9.1 Dockerfile

```
1 #####
2 # Raspberry Pi 4
3 # Raspberry Pi OS Bullseye (32-bit)
4 #####
5
6 FROM arm32v7/debian:bullseye
7
8 ENV DEBIAN_FRONTEND=noninteractive
9
10 WORKDIR /app
11
12 #####
13 # Install Python
14 #####
15
16 RUN apt-get update && \
17     apt-get install -y \
18         python3 \
19         python3-pip \
20         python3-dev \
21         python3-setuptools \
22         python3-wheel \
23         && rm -rf /var/lib/apt/lists/*
24
25 #####
26 # Add Raspberry Pi Repository
27 #####
28
29 RUN apt-get update && \
30     apt-get install -y wget gnupg ca-certificates
31
32 RUN wget https://archive.raspberrypi.org/debian/raspberrypi.gpg.key -O
/tmp/rpi.gpg && \
33     apt-key add /tmp/rpi.gpg
34
35 RUN echo "deb http://archive.raspberrypi.org/debian bullseye main" >
/etc/apt/sources.list.d/raspi.list
36
37 #####
38 # Install Raspberry Pi Camera Packages
39 #####
40
41
42 RUN apt-get update && \
43     apt-get install -y \
44         python3-picamera2 \
45         python3-libcamera \
46         python3-opencv \
47         python3-pyzbar \
48         libcamera0 \
49         libcamera-apps \
50         libcamera-tools \
```

```
51         libzbar0 \
52         libgtk-3-0 \
53         libcanberra-gtk3-module \
54         x11-apps \
55         && rm -rf /var/lib/apt/lists/*
56
57
58
59 #####
60 # Copy source
61 #####
62
63 COPY . .
64
65 CMD ["python3", "barcode_reader.py"]
```

9.2 docker-compose.yml

```
1  version: "3.9"
2
3  services:
4
5    barcode_nowindow:
6      image: barcode_nowindow:v1
7      build: .
8
9      container_name: barcode_nowindow
10
11     privileged: true
12
13     network_mode: host
14
15     stdin_open: true
16
17     tty: true
18
19     restart: "no"
20
21     environment:
22       - DISPLAY=${DISPLAY}
23
24     volumes:
25       - /tmp/.X11-unix:/tmp/.X11-unix
26       - /dev:/dev
27       - /run/udev:/run/udev:ro
28       - /run/dbus:/run/dbus
29
30     devices:
31       - /dev/video0
32
```

9.3 barcode_reader.py

```
1  from picamera2 import Picamera2
2  import cv2
3  from pyzbar.pyzbar import decode
4
5  # -----
6  # Camera Initialization
7  # -----
8
9  picam2 = Picamera2()
10
11  config = picam2.create_preview_configuration(
12      main={"size": (640, 480)}
13  )
14
15  picam2.configure(config)
16  picam2.start()
17
18  print("Camera started.")
19  print("Press q to quit.")
20
21  last_barcode = ""
22
23  # -----
24  # Main Loop
25  # -----
26
27  while True:
28
29      frame = picam2.capture_array()
30
31      # Picamera2 returns RGB
32      frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
33
34      barcodes = decode(frame)
35
36      for barcode in barcodes:
37
38          x, y, w, h = barcode.rect
39
40          # Draw green rectangle
41          '''
42          cv2.rectangle(
43              frame,
44              (x, y),
45              (x + w, y + h),
46              (0, 255, 0),
47              2
48          )
49          '''
50
51          barcode_data = barcode.data.decode("utf-8")
52          barcode_type = barcode.type
53
```

```

54         # Display text above barcode
55         '''
56         text = f"{barcode_type}: {barcode_data}"
57
58         cv2.putText(
59             frame,
60             text,
61             (x, y - 10),
62             cv2.FONT_HERSHEY_SIMPLEX,
63             0.6,
64             (0, 255, 0),
65             2
66         )
67         '''
68
69         # Avoid printing the same barcode repeatedly
70         if barcode_data != last_barcode:
71
72             last_barcode = barcode_data
73
74             print()
75             print("=====")
76             print("Barcode Detected")
77             print("-----")
78             print("Type :", barcode_type)
79             print("Data :", barcode_data)
80             print("=====")
81
82         #cv2.imshow("Barcode Reader", frame)
83         '''
84         key = cv2.waitKey(1)
85
86         if key == ord('q'):
87             break
88         '''
89
90     picam2.stop()
91
92     #cv2.destroyAllWindows()

```